

CSC1109 – Data (Analytics) at Speed and Scale

Alessandra Mileo

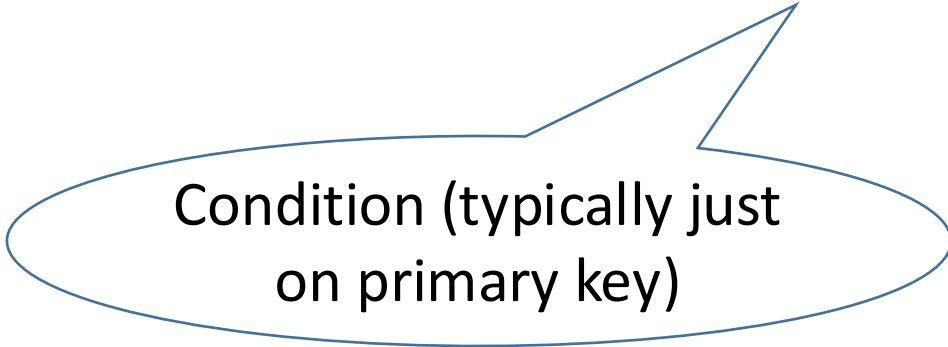
alessandra.mileo@dcu.ie

Hive Basics: Advanced Queries

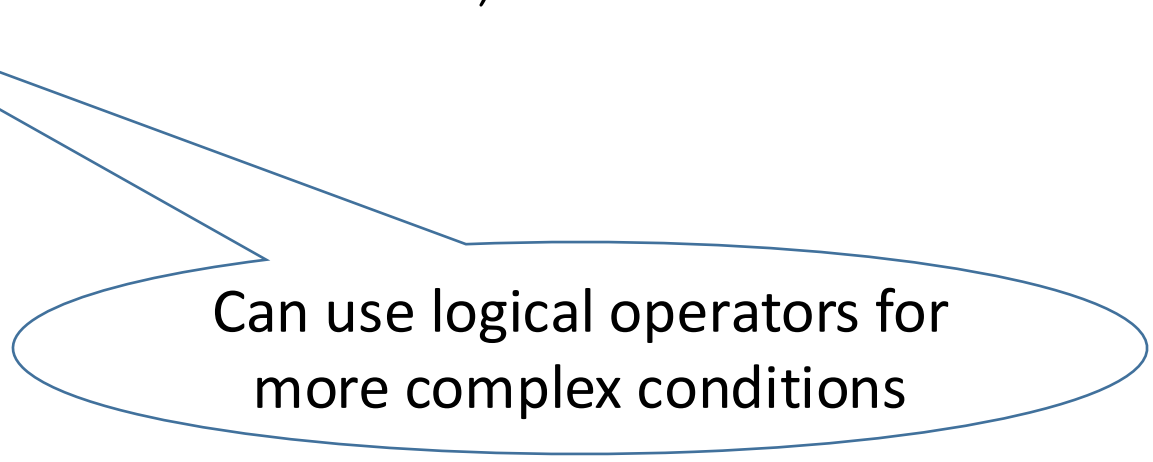
- Simple Select queries
- Hive Functions (Simple and Aggregated)
- Advanced Queries
 - Join
 - Sampling (with or without subqueries)
 - Subqueries

JOIN (recap from SQL/Databases)

```
SELECT *  
  FROM a  
  JOIN b  
  ON (a.id = b.id AND a.usr = b.usr)
```



Condition (typically just
on primary key)



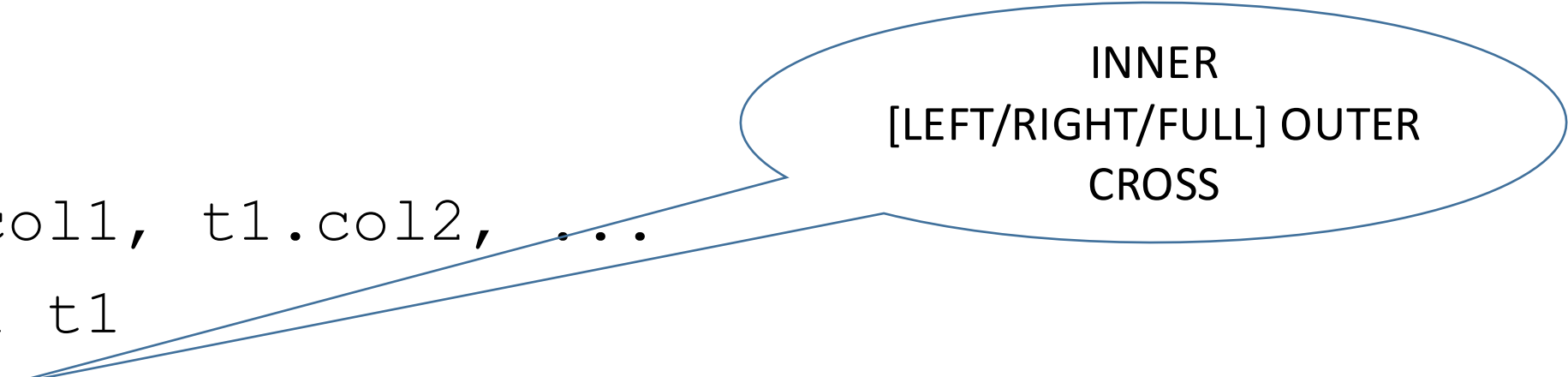
Can use logical operators for
more complex conditions

JOIN

```
SELECT column1, column2, ...  
FROM table1  
[JOIN_TYPE] JOIN table2  
ON (table1.key1 = table2.key2)  
[WHERE condition]
```

JOIN

```
SELECT t1.col1, t1.col2, ...  
FROM table1 t1  
[JOIN_TYPE] JOIN table2 t2  
ON (t1.key1 = t2.key2)  
[WHERE condition]
```



INNER
[LEFT/RIGHT/FULL] OUTER
CROSS

INNER JOIN

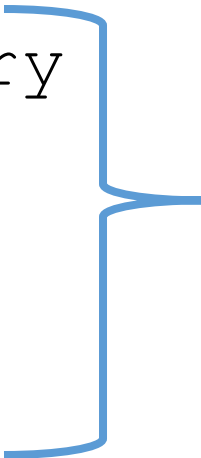
employees

id	name	department
1	John	HR
2	Mike	IT
3	Anna	Finance

salaries

emp_id	salary
1	50000
3	70000

```
SELECT a.id, a.name, b.salary
FROM employees a
INNER JOIN salaries b
ON a.id = b.emp_id;
```



id	name	salary	
1	John	50000	
3	Anna	70000	

LEFT OUTER JOIN

employees

id	name	department
1	John	HR
2	Mike	IT
3	Anna	Finance

salaries

emp_id	salary
1	50000
3	70000

```
SELECT a.id, a.name, b.salary
FROM employees a
LEFT OUTER JOIN salaries b
ON a.id = b.emp_id;
```

id	name	salary
1	John	50000
2	Mike	NULL
3	Anna	70000

RIGHT OUTER JOIN

employees

id	name	department
1	John	HR
2	Mike	IT
3	Anna	Finance

salaries

emp_id	salary
1	50000
3	70000

```
SELECT a.id, a.name, b.salary
FROM employees a
RIGHT OUTER JOIN salaries b
ON a.id = b.emp_id;
```

id	name	salary
1	John	50000
3	Anna	70000

FULL OUTER JOIN

employees

id	name	department
1	John	HR
2	Mike	IT
3	Anna	Finance

salaries

emp_id	salary
1	50000
3	70000
4	60000

```
SELECT a.id, a.name, b.salary
FROM employees a
FULL OUTER JOIN salaries b
ON a.id = b.emp_id;
```

id	name	salary
1	John	50000
2	Mike	NULL
3	Anna	70000
NULL	NULL	60000

Joining multiple tables

```
SELECT a.id, a.department, b.salary, c.building
FROM employees a
FULL OUTER JOIN salaries b
ON a.id = b.emp_id
LEFT OUTER JOIN income c
ON a.department = c.department;
```

Joins are always left associative and not commutative

CROSS JOIN

```
SELECT s.student_name, c.course_name  
FROM students s  
CROSS JOIN courses c;
```

- Cartesian product
- One row per student per course
- Computationally expensive (limit use unless needed)

Partitions in Hive

- Create table (with partitions):

```
CREATE TABLE sales (  
    sales_id INT,  
    product_name STRING,  
    amount DOUBLE,  
) PARTITIONED BY (state STRING);
```

- Add data into partitioned table

```
INSERT INTO TABLE sales PARTITION (state='CA')  
VALUES (1, 'Product_name', 100.0);
```

Bucketing (hash partitioning) - CREATE

- Create table (partitioned into a number of buckets)

```
CREATE TABLE customers (  
    customer_id INT,  
    name STRING,  
    email STRING  
)  
CLUSTERED BY (customer_id) INTO 32 BUCKETS  
STORED AS [file_format];
```

Note: can combine partitioning and bucketing (they are complementary):

<https://www.sparkcodehub.com/hive/partitions/partition-vs-bucketing>

Bucketing (hash partitioning) - INSERT

- Insert data by enforcing bucketing:

```
SET hive.enforce.bucketing=true;
```

```
INSERT INTO TABLE customers  
SELECT customer_id, name, email  
FROM raw_customers;
```

Note: each row is assigned to a bucket based on the result of:

$$\text{hash}(\text{customer_id}) \bmod 32$$

Sampling

- Create bucketed table
- Use `TABLESAMPLE` clause in any `FROM` clause within `SELECT`

Block Sampling:



Alias

```
SELECT * FROM table TABLESAMPLE (n PERCENT) s;  
SELECT * FROM table TABLESAMPLE (10 ROWS) ;
```

Bucket Sampling

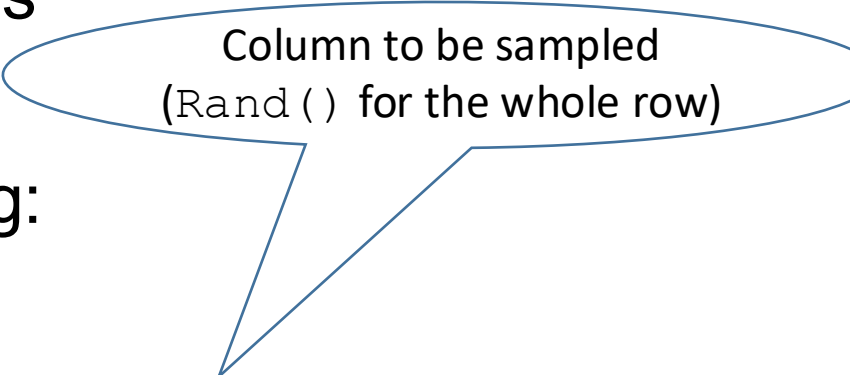
Let's assume `table` was created with 32 clusters

```
CLUSTERED BY (id) INTO 32 BUCKETS
```

The following `SELECT` applies bucketed sampling:

```
SELECT * FROM table
```

```
TABLESAMPLE (BUCKET x OUT OF y [ON col_i]) s;
```



Column to be sampled
(Rand () for the whole row)

1. `TABLESAMPLE (BUCKET 3 OUT OF 16)` returns clusters 3 and 19
2. `TABLESAMPLE (BUCKET 2 OUT OF 8)` returns clusters 2, 10, 19, 27

Subqueries

- Used within an existing `SELECT` query

When I use nested `SELECT` queries within

- `SELECT ... (SELECT ...)`
- `FROM (SELECT ...) clause` } Also called derived table
- `WHERE/HAVING [condition with subquery] clause`
- `WHERE EXIST (SELECT ...) clause`
- `WHERE NOT EXIST (SELECT ...) clause` } Inner query checks existence

Subqueries

Scalar subquery

- Values of the outer query ***are not used*** in the inner query
- Returns one column value for one row
- Example: find products with sale amount greater than the average

```
SELECT product_id
FROM sales
WHERE sale_amount > (SELECT AVG(sale_amount)
                     FROM sales);
```

Subqueries

Correlated subquery

- Values from the outer query ***are used*** in the inner query
- Runs once for each row processed by the outer query
- Example: find the name of products that have been sold

```
SELECT product_name
FROM products
WHERE product_id = ANY (SELECT product_id FROM sales
                        WHERE products.product_id = sales.product_id);
```

JOINS with Subqueries

- Subqueries can be expensive so limit usage
- Translating subqueries as JOINS can be more optimal

See some examples here

<https://learnsql.com/blog/converting-subqueries-joins/>

LAB: Where are we?

Hive LAB

<https://csc1109.readthedocs.io/en/latest/lab2/>